

3章2節 コン ピュータの仕組み と動作

加納 寛之

2026-09-15

今日の学習内容

■ 3章 コンピュータとプログラミング

■ 2節 コンピュータの仕組みと動作

1. ハードウェアとソフトウェア
2. 数値の計算
3. 演算の仕組み

1 ハードウェアとソフトウェア

ハードウェアとソフトウェア

- コンピュータは、 **ハードウェア**と**ソフトウェア**で構成される
- **ハードウェア** —— コンピュータの物理的な機械の部分
- **ソフトウェア** —— その機械を動かすプログラム

コンピュータの構成

- **入力装置** —— キーボード・マウスなどからデータやプログラムを入力
- **主記憶装置**（メインメモリ）や**補助記憶装置**（HDD・SSDなど）に保存
- **演算装置**によって計算を行い、**出力装置**（ディスプレイ・プリンタなど）へ出力
- これらの装置を制御するのが**制御装置**
- **演算装置**と**制御装置**を合わせて**中央処理装置（CPU）**という

クロック周波数とGPU

- **クロック周波数** —— 1秒間の信号の回数。CPUの性能を表す尺度の一つ
 - ほかの条件が同じなら、クロック周波数が高いほど処理速度が速い
- **GPU** (Graphics Processing Unit)
 - 画像処理・画像の表示・3Dグラフィックスなどの計算や処理を行う半導体チップ

情報機器の接続

- 情報機器を相互に接続する規格を**インタフェース**という
- 周辺機器との接続：**USB**インタフェースや無線を利用
- 有線LAN：LANケーブルで**イーサネット**規格を使用
 - **ハブ** —— 複数機器を有線接続する集線装置（スイッチとも呼ばれる）
 - **ルータ** —— 異なるネットワークを相互接続する機器
- 無線LAN：**IEEE 802.11**の規格を利用
- **Bluetooth** —— 2.4GHz帯を使う近距離無線通信（数m程度）

ソフトウェアの種類

- **基本ソフトウェア**：オペレーティングシステム (OS)
- **応用ソフトウェア** (アプリケーションソフトウェア／アプリ)
 - 文書作成・表計算・プレゼンテーション・Webブラウザ・画像処理など
- OSは**デバイスドライバ**を追加することで、さまざまな周辺機器に対応できる
 - アプリ利用者はハードウェアの違いを意識せずに作業できる

OSの役割

- OSはハードウェアと応用ソフトウェアの仲介を行う
- 主な役割
 - タスク管理・メモリ管理・入出力管理
 - ファイル管理・リソース管理・ユーザ管理
- **GUI** —— アイコンやウィンドウで操作する環境
 - 対して文字で命令を入力する環境を**CUI**/**CLI**という
- **API** —— アプリからハードウェアや機能呼び出すためのインタフェース
 - 見た目や操作の統一・互換性の維持に役立つ
- **Webアプリ** —— ブラウザ上で動き、異なるOSでも同じアプリを使える

コンピュータの動作

- 主記憶装置にはデータや命令が保存されている
- CPU内にはデータや命令を一時的に高速保存する**レジスタ**がある
- 主記憶装置とレジスタの間でデータ・命令をやり取りし、プログラムを実行する
- **プログラムカウンタ**もレジスタの一つで、次に実行する命令の番地を指す

CPUの基本動作（加算の流れ）

- 「3+5」を計算するときのCPU内の流れ（イメージ）
 - 命令「読み出し」→ データ「3」をレジスタAへ
 - 命令「読み出し」→ データ「5」をレジスタBへ
 - 命令「加算」→ 結果「8」をレジスタCへ
 - 命令「保存」→ 主記憶装置へ結果を書き戻す
 - 命令「終了」
- 各命令の実行後、プログラムカウンタが次の番地に更新される

2 数値の計算

2進数の加算と減算

- 2進数の加算・減算は、10進数と同じく桁ごとに計算する
- 加算は**繰り上げ**（桁上げ），減算は**繰り下げ**（桁借り）を行う
- 計算するときは，桁数（ビット数）を揃えてから行う

負の数の表現

- コンピュータでは、負の数をもとの数の**補数**として表現する
- ここでの補数：2進数のある数に対し、「足すと1桁増える最も小さな数」
 - 例：4ビットの0100 (2) の補数は1100 (2)
 - $0100 + 1100 = 10000$ (2桁上がり)

補数を使った減算

- 補数を使うと、減算を**すべて加算**で行うことができる
- メリット：減算用の回路を用意しなくてよく、演算回路が簡単になる
- 手順のイメージ
 - 引く数の補数を求める
 - 補数を足し、桁上がりを無視する → 減算と同じ結果になる

コンピュータでの実数の表現

- 2進数の小数：2の-1乗 (0.5) , 2の-2乗 (0.25) , 2の-3乗 (0.125) ...の和
 - 例： $0.101\ (2) = 0.5 + 0.125 = 0.625\ (10)$
- 10進数→2進数の小数変換：小数点以下を0になるまで2倍する
- **固定小数点数** —— 小数点の位置を固定。表現できる数値の範囲が狭い
- **浮動小数点数** —— 仮数×基数の指数 で表す（本書はIEEE 754）

浮動小数点数 (32ビット)

- 32ビット浮動小数点数の構成
 - **符号部** : 1ビット (正=0, 負=1)
 - **指数部** : 8ビット (バイアス値127を加えて調整)
 - **仮数部** : 23ビット (最上位の1を省略して格納)
- IEEE 754のほかの精度
 - 16ビット (半精度) ・ 64ビット (倍精度) など

コンピュータでの計算と誤差

- ビット数が有限のため、理論値と計算結果がずれることがある → **誤差**
- **丸め誤差** —— 有限桁で四捨五入・切り捨て・切り上げする**丸め処理**による誤差
 - 例：0.1 (10) は2進数では循環小数になる
- **桁落ち誤差** —— 近い大きさの小数どうしの減算で有効数字が減る現象
- **オーバーフロー** —— 演算結果が表示範囲の最大値を超えること
- **アンダーフロー** —— 演算結果が表示範囲の最小値より小さくなること
- これらによる誤差を**桁あふれ誤差**という

3 演算の仕組み

論理回路

- コンピュータは「0」と「1」の信号で演算や制御を行う
- その回路を**論理回路**という
- 論理回路は**論理演算**を実行し、結果を「1」または「0」で出力する
- 基本の3つ：**AND・OR・NOT**
 - この3つの組み合わせですべての計算ができる
- ほかにNAND・NOR・XOR (EOR) などもある

論理積回路（AND回路）

- 二つの入力と一つの出力をもつ
- **両方の入力がともに1のときだけ**，出力が1になる
- 直列スイッチのイメージ：両方ONのときだけランプが点灯
- 入力と出力の関係を示す表を**真理値表**という
- 電子回路の設計図を**回路図**といい，部品を図記号で表す

AND回路の真理値表

■ 入力A・B → 出力L

■ 0, 0 → 0

■ 0, 1 → 0

■ 1, 0 → 0

■ 1, 1 → 1

論理和回路 (OR回路)

- 二つの入力と一つの出力をもつ
- **入力のいずれか一方が1であれば**, 1を出力する
- 並列スイッチのイメージ: どちらか一方でもONならランプが点灯

OR回路の真理値表

■ 入力A・B → 出力L

■ 0, 0 → 0

■ 0, 1 → 1

■ 1, 0 → 1

■ 1, 1 → 1

否定回路 (NOT回路)

- 一つの入力と一つの出力をもつ
- **入力した信号を反転**した値を出力する
 - $0 \rightarrow 1$ $1 \rightarrow 0$

NOT回路の真理値表

■ 入力A → 出力L

■ $0 \rightarrow 1$

■ $1 \rightarrow 0$

半加算回路

- AND・OR・NOTを組み合わせた回路の例が**半加算回路**
- 出力
 - **C** (Carry) —— 桁上げ
 - **S** (Sum) —— 1桁目の和
- 半加算回路は、2進数1桁の加算を表している
- 論理回路を組み合わせることで、演算ができる

半加算回路の真理値表

■ 入力A・B → 出力C（桁上げ）・S（和）

■ 0, 0 → C=0, S=0

■ 0, 1 → C=0, S=1

■ 1, 0 → C=0, S=1

■ 1, 1 → C=1, S=0

■ これは2進数1桁の加算結果と一致する

まとめ

- **ハードウェア／ソフトウェア**：機械部分と、それを動かすプログラム
- **CPU**：演算装置＋制御装置。レジスタとプログラムカウンタで命令を実行
- **インタフェース**：USB・イーサネット・IEEE 802.11・Bluetooth
- **OS**：ハードウェアとアプリの仲介。GUI／CUI, API, デバイスドライバ
- **補数**：負の数の表現。減算を加算に置き換えられる
- **浮動小数点数**：符号部・指数部・仮数部（IEEE 754）
- **誤差**：丸め誤差・桁落ち誤差・桁あふれ誤差（オーバーフロー／アンダーフロー）
- **論理回路**：AND・OR・NOT。組み合わせで半加算などの演算が可能

次回予告

4章 アルゴリズムとプログラミングー 1節 アルゴリズムとプログラミング言語 に進みます